FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Andreu Liam C. Gamboa
Section:	AN43
Professor:	Crisola G. Tan
Date:	10/12/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

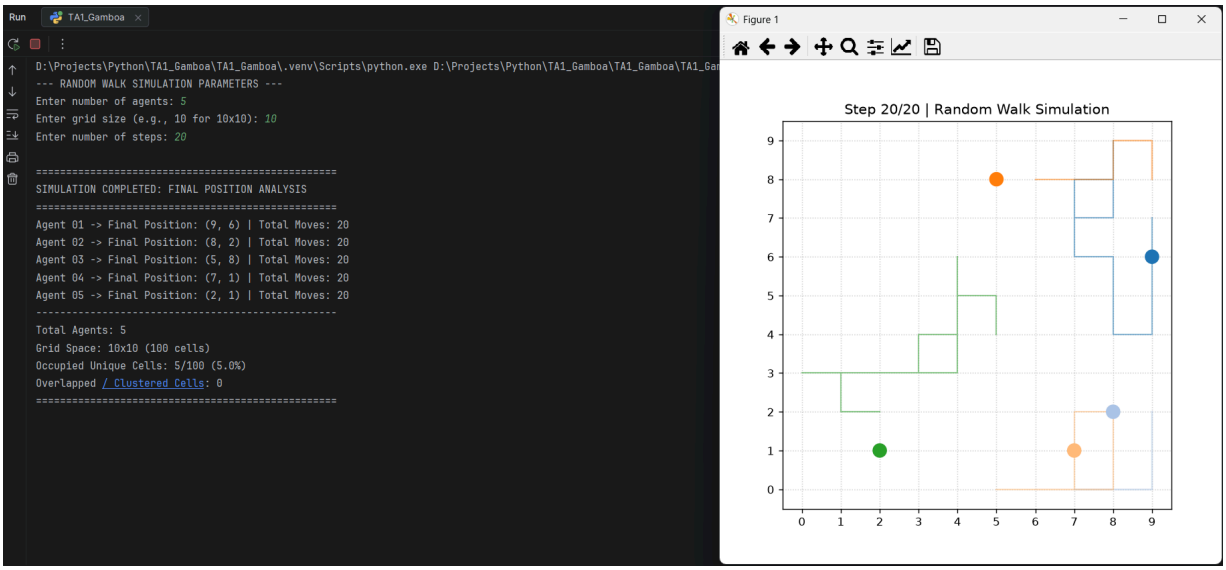
- **More agents** (e.g., 10 or 20)
- **Smaller/larger grid sizes**
- **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

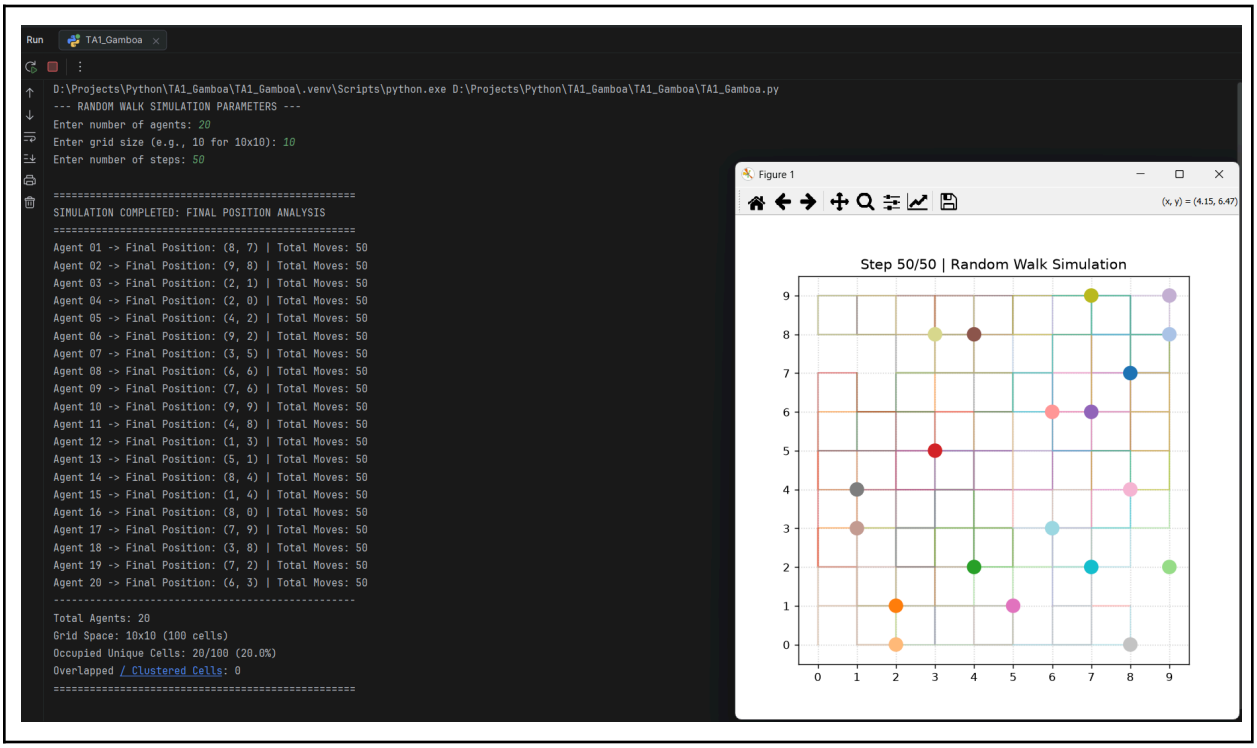
Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Scenario 1



Scenario 2



Modified (Updated Code - Laboratory Tasks) ⇩

Source Code

```
"""
CS0065 - Intelligent Systems
Technical Assessment 1: Random Walk Simulation using AgentPy
File: TA1_Gamboa.py
"""

import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import Counter


# ================= 1. Agent Definition =================
class SmartWalker(ap.Agent):
    def setup(self):
        self.position = (0, 0)
```

```
        self.path = []  # Task 1: Track movement trajectory

    def step(self):
        # Record current position before stepping
        self.path.append(self.position)

        # 4 possible directions (Von Neumann neighborhood)
        possible_directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        random.shuffle(possible_directions)

        # Task 2: Preferred movement rule (avoid occupied cells)
        occupied_positions = {other.position for other in self.model.agents if other != self}

        chosen_pos = None
        for dx, dy in possible_directions:
            candidate_x = max(0, min(self.model.p.grid_size[0] - 1, self.position[0] + dx))
            candidate_y = max(0, min(self.model.p.grid_size[1] - 1, self.position[1] + dy))
            candidate = (candidate_x, candidate_y)

            if candidate not in occupied_positions:
                chosen_pos = candidate
                break

        # Fallback to bounded step if surrounded
        if chosen_pos is None:
            dx, dy = random.choice([(1, 0), (-1, 0), (0, 1), (0, -1)])
            chosen_pos = (
                max(0, min(self.model.p.grid_size[0] - 1, self.position[0] + dx)),
                max(0, min(self.model.p.grid_size[1] - 1, self.position[1] + dy))
            )

        self.position = chosen_pos


# ================ 2. Model Definition ================
class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, SmartWalker)
        total_cells = self.p.grid_size[0] * self.p.grid_size[1]
        all_coords = [(x, y) for x in range(self.p.grid_size[0]) for y in range(self.p.grid_size[1])]

        if self.p.agents <= total_cells:
            starting_coords = random.sample(all_coords, self.p.agents)
```

```
        else:
            starting_coords = [random.choice(all_coords) for _ in range(self.p.agents)]

        for agent, coord in zip(self.agents, starting_coords):
            agent.position = coord
            agent.path.append(coord)

    def step(self):
        self.agents.step()

    def end(self):
        # Task 3: Analyze and output final coordinates
        print("\n" + "=" * 50)
        print("SIMULATION COMPLETED: FINAL POSITION ANALYSIS")
        print("=" * 50)
        final_positions = [agent.position for agent in self.agents]
        coord_counts = Counter(final_positions)

        for idx, agent in enumerate(self.agents):
            print(f"Agent {idx + 1:02d} -> Final Position: {agent.position} | Total Moves: {len(agent.path) - 1}")

        unique_tiles = len(coord_counts)
        total_tiles = self.p.grid_size[0] * self.p.grid_size[1]
        print("-" * 50)
        print(f"Total Agents: {self.p.agents}")
        print(f"Grid Space: {self.p.grid_size[0]}x{self.p.grid_size[1]} ({total_tiles} cells)")
        print(f"Occupied Unique Cells: {unique_tiles}/{total_tiles} ({unique_tiles / total_tiles * 100:.1f}%)")
        print(f"Overlapped / Clustered Cells: {sum(1 for v in coord_counts.values() if v > 1)}")
        print("=" * 50 + "\n")


# ================= 3. Simulation & Animation =================
def run_simulation():
    print("--- RANDOM WALK SIMULATION PARAMETERS ---")
    num_agents = int(input("Enter number of agents: "))
    grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
    num_steps = int(input("Enter number of steps: "))

    parameters = {
        'agents': num_agents,
        'grid_size': (grid_size, grid_size),
        'steps': num_steps
    }
```

```
model = RandomWalkModel(parameters)
model.setup()

fig, ax = plt.subplots(figsize=(6, 6))
colors = plt.cm.tab20.colors

path_lines = [
    ax.plot([], [], color=colors[i % len(colors)], linestyle='-', linewidth=1.2, alpha=0.5)[0]
    for i in range(parameters['agents'])
]
scat = ax.scatter([], [], s=130, edgecolors='black', zorder=3)

def init():
    ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
    ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
    ax.set_xticks(range(model.p.grid_size[0]))
    ax.set_yticks(range(model.p.grid_size[1]))
    ax.grid(True, linestyle=":", alpha=0.6)
    return path_lines + [scat]

def update(frame):
    model.step()
    for i, agent in enumerate(model.agents):
        xs = [pos[0] for pos in agent.path]
        ys = [pos[1] for pos in agent.path]
        path_lines[i].set_data(xs, ys)

    current_x = [agent.position[0] for agent in model.agents]
    current_y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(current_x, current_y)))
    scat.set_color([colors[i % len(colors)] for i in range(model.p.agents)])
    ax.set_title(f"Step {frame + 1}/{model.p.steps} | Random Walk Simulation")

    if frame == model.p.steps - 1:
        model.end()

    return path_lines + [scat]

# Storing animation in anim so it is retained during execution
anim = animation.FuncAnimation(
    fig,
    update,
```

```
        frames=model.p.steps,
        init_func=init,
        blit=False,
        repeat=False
    )
    plt.show()


if __name__ == "__main__":
    run_simulation()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1
When agent count approaches grid capacity, spatial saturation occurs. In unconstrained models, multiple agents overlap on identical cells. In collision-avoidance models, surrounding cells become blocked, causing movement gridlock and severely restricting agent dispersion.

2
Increasing steps allows agents to diffuse further from their initial spawn points. Because random walk displacement scales with $\sqrt{t}$, longer runs distribute agents more uniformly across the bounded grid.

3
No, agents never move outside the grid. The code explicitly restricts movement using boundary clamping (max(0, min(grid_size - 1, coordinate + direction))), which limits all positions within valid matrix boundaries (0 to $grid_size - 1$).

4
● Molecular Diffusion: Simulates Brownian particle motion spreading from high concentration toward uniform dispersion. ● Pedestrian/Crowd Dynamics: Models personal space buffers and room evacuation patterns through constrained pathways. ● Epidemic Modeling: Can track infection transmission when agents occupy adjacent or identical coordinates. ● Animal Foraging: Models random exploratory search patterns of animals or insects within a territory.